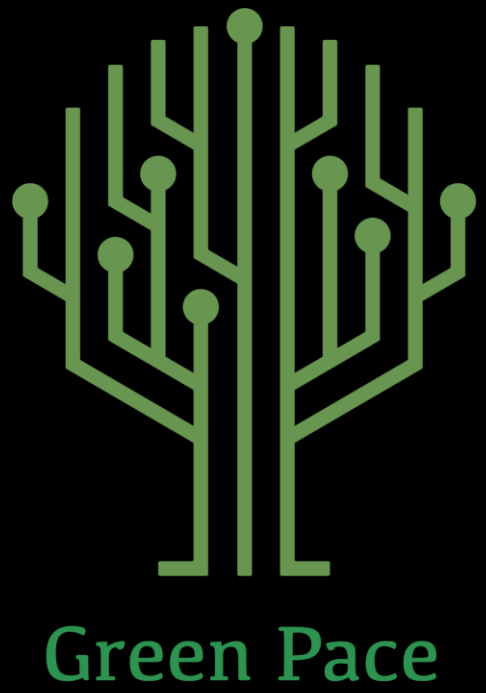
Green Pace

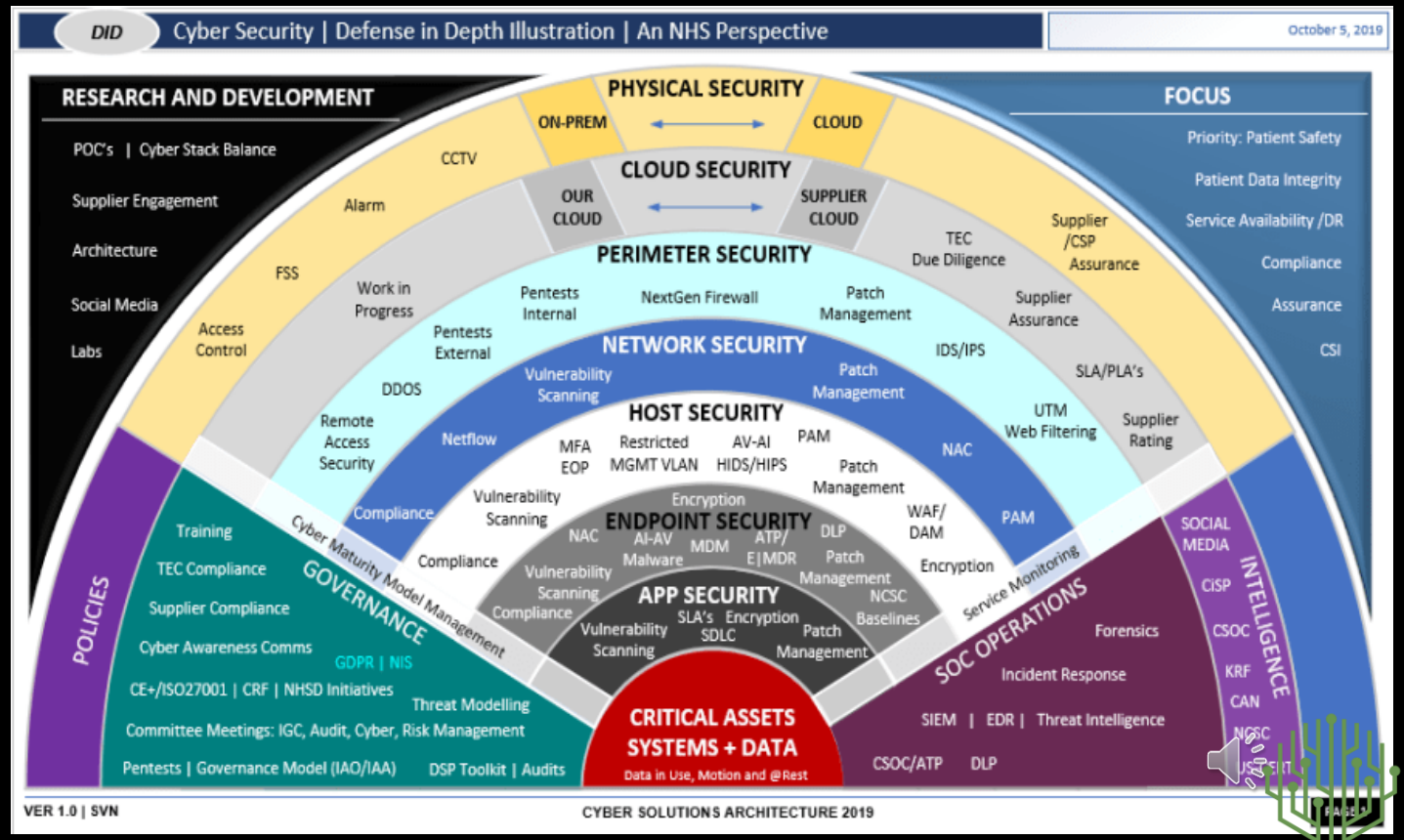
Security Policy Presentation

Developer: Sabrina Ozburn



OVERVIEW: DEFENSE IN DEPTH

- Define core security principles, coding standards, Triple A, and data encryption standards.
- Dipping into multiple layers in depth and defense
- Avoiding security vulnerabilities



Green Pace

THREATS MATRIX

The threats are organized by similarity and priority level. Their likelihood determines how frequently they should be reviewed and, when possible, mitigated through automation. High-priority threats must be addressed promptly, as they pose the greatest risk of disruption to the product and overall security.

Likely STD-003-CPP STD-005-CPP STD-008-CPP	**Priority** STD-002-CPP STD-004-CLG
Low priority STD-007-CPP STD-010-CPP	**Unlikely** STD-001-CPP STD-006-CLG STD-009-CPP

Green Pace

10 PRINCIPLES

1. Validate Input Data
   - Data Type
   - String Correctness
2. Heed Complier warnings
   - Data Type
   - Data Value
   - String Correctness
   - Memory Protection
3. Architect and Design for Security Policies
   - Assertions
   - Exceptions
   - Memory Protection
4. Keep It Simple
   - Data Type
   - Data Value
   - SQL Injection
   - Memory Protection
   - Assertions
   - Exceptions
   - String Correctness
5. Default Deny
   - SQL Injection
   - Exceptions
   - Memory Protection
6. Adhere to the Principle of Least Privilege
   - String Correctness
7. Sanitize Data Sent to Other Systems
8. Practice Defense in Depth
   - String Correctness
   - Exceptions
   - Exceptions
9. Use Effective Quality Assurance Techniques
   - Data Value
   - Memory Protection
   - Assertions
10. Adopting a Secure Coding Standard
    - Data Type
    - Data Value
    - String Correctness
    - SQL Injection
    - Memory Protection
    - Assertions
    - Exceptions

Green Pace

CODING STANDARDS

When ranking vulnerabilities, it is essential to determine which issues would cause the greatest harm to the product and prioritize them accordingly. Based on established coding standards, vulnerabilities related to Data Types, SQL Injection, and Memory Protection are likely to have the most severe impact and should be addressed first.

After assessing potential impact, the next step is to evaluate the likelihood of occurrence. Errors involving Data Types/Values, and String Correctness are among the most common in C/C++ development, making them a high priority for review and remediation.

Finally, implementing thorough testing practices and maintaining well-structured code can help prevent future issues while also identifying existing ones. Utilizing Assertions and Exceptions supports more efficient coding and strengthens overall testing efforts.

- Data Type
- Data Value
- String Correctness
- String Correctness
- SQL Injection
- Memory Protection
- Memory Protection
- Assertions
- Exceptions
- Exceptions

Green Pace

ENCRYPTION POLICIES

Encryption at rest:

- Helps protect data stored on a disk
- Keeps data secure from possible attacks if hardware is accessed by others or if an attacker gains access to the personal system.

Encryption in flight:

- Protects data confidentiality between client and a server
- Prevents interception of data by attackers through encryption

Encryption in use:

- Ensures continuous protection of data at all stages and locations
- Prevents data from being unsecured
- Important policy for continuously safeguarding data and reducing the risk of breaches

Green Pace

TRIPLE-A POLICIES

Authentication: Used to verify the identity of a user

- Can be done though usernames, passwords, and biometrics
- Ensures that only authorized users have access to resources, keeping information secure

Authorization: Determines roles, privileges and access levels after authentication

- Controls access to functions, resources and data
- Ensures that only authorized users perform specific actions, keeping information secure.

Accounting: Tracks resources usage and user actions of authenticated and authorized users

- Logs information such as time of access, actions performed and user identity
- Helps identify unauthorized access attempts and potential security breaches.

Green Pace

Unit Testing
Adding a value

```
// TODO: Create a test to verify adding a single value to an empty collection
TEST_F(CollectionTest, CanAddToEmptyVector)
{
    // is the collection empty?
    ASSERT_TRUE(collection->empty());

    // if empty, the size must be 0
    ASSERT_EQ(collection->size(), 0);

    // Add entry
    add_entries(1);

    // is the collection still empty?
    EXPECT_FALSE(collection->empty());

    // if not empty, what must the size be?
    EXPECT_EQ(collection->size(), 1);

}
```

```
[ RUN      ] CollectionTest.CanAddToEmptyVector
[       OK ] CollectionTest.CanAddToEmptyVector (0 ms)
```

```
// TODO: Create a test to verify adding five values to collection
TEST_F(CollectionTest, CanAddFiveValuesToVector)
{
    // Add 5 entries
    add_entries(5);

    // Check that it's 5 entries
    ASSERT_EQ(collection->size(), 5);

}
```

```
[ RUN      ] CollectionTest.CanAddFiveValuesToVector
[       OK ] CollectionTest.CanAddFiveValuesToVector (0 ms)
```

Green Pace

Unit Testing
Max Size

```
// TODO: Create a test to verify that max size is greater than or equal to size for 0, 1, 5, 10 entries

TEST_F(CollectionTest, MaxSizeGreaterOrEqualToSize)
{
    // Add entries
    add_entries(11);

    //Verify size is greater than or equal to entries
    ASSERT_TRUE(collection->max_size() >= 0);
    ASSERT_TRUE(collection->max_size() >= 1);
    ASSERT_TRUE(collection->max_size() >= 5);
    ASSERT_TRUE(collection->max_size() >= 10);

}
```

```
[ RUN      ] CollectionTest.MaxSizeGreaterOrEqualToSize
[       OK ] CollectionTest.MaxSizeGreaterOrEqualToSize (0 ms)
```

Green Pace

Unit Testing
Check Capacity

```
// TODO: Create a test to verify that capacity is greater than or equal to size for 0, 1, 5, 10 entries

TEST_F(CollectionTest, CapacityGreaterOrEqualToSize)
{
    //  Add entries
    add_entries(11);

    // Verify capacity is greater then or equal to the entries
    ASSERT_TRUE(collection->capacity() >= 0);
    ASSERT_TRUE(collection->capacity() >= 1);
    ASSERT_TRUE(collection->capacity() >= 5);
    ASSERT_TRUE(collection->capacity() >= 10);

}
```

```
[ RUN      ] CollectionTest.CapacityGreaterOrEqualToSize
[       OK ] CollectionTest.CapacityGreaterOrEqualToSize (0 ms)
```

Green Pace

Unit Testing Resizing

```
// TODO: Create a test to verify resizing increases the collection

TEST_F(CollectionTest, ResizingIncreasesCollection)
{
    // Add entries
    add_entries(10);

    // Verifying the increase of the collection
    size_t previousSize = collection->size();
    collection->resize(20);
    ASSERT_TRUE(collection->size() > previousSize);

}
```

```
// TODO: Create a test to verify resizing decreases the collection

TEST_F(CollectionTest, ResizingDecreasesCollection)
{
    // Add entries
    add_entries(20);

    // Verifying the decrease of the collection
    size_t previousSize = collection->size();
    collection->resize(1);
    ASSERT_TRUE(collection->size() < previousSize);

}
```

```
// TODO: Create a test to verify resizing decreases the collection to zero

TEST_F(CollectionTest, ResizingDecreasesToZero)
{
    // Add entries
    add_entries(20);

    // Verify decrease of collection to zero
    size_t previousSize = collection->size();
    collection->resize(0);
    ASSERT_TRUE(collection->size() == 0);

}
```

```
[ RUN      ] CollectionTest.ResizingIncreasesCollection
[       OK ] CollectionTest.ResizingIncreasesCollection (0 ms)
[ RUN      ] CollectionTest.ResizingDecreasesCollection
[       OK ] CollectionTest.ResizingDecreasesCollection (0 ms)
[ RUN      ] CollectionTest.ResizingDecreasesToZero
[       OK ] CollectionTest.ResizingDecreasesToZero (0 ms)
```

Green Pace

Unit Testing
Clearing & Erasing

```
// TODO: Create a test to verify clear erases the collection

TEST_F(CollectionTest, ClearErasesCollection)
{
    // Add entries
    add_entries(15);

    // Verify clear erases the collection
    collection->clear();
    ASSERT_TRUE(collection->size() == 0);
}
```

```
[ RUN      ] CollectionTest.ClearErasesCollection
[       OK ] CollectionTest.ClearErasesCollection (0 ms)
```

```
// TODO: Create a test to verify erase(begin,end) erases the collection

TEST_F(CollectionTest, EraseRangeErasesCollection)
{
    // Add entries
    add_entries(20);

    //  Verify it erases the begining to end collection
    collection->erase(collection->begin(), collection->end());
    ASSERT_TRUE(collection->size() == 0);

}
```

```
[ RUN      ] CollectionTest.EraseRangeErasesCollection
[       OK ] CollectionTest.EraseRangeErasesCollection (0 ms)
```

Green Pace

Unit Testing
Reserve Increase

```
// TODO: Create a test to verify reserve increases the capacity but not the size of the collection

TEST_F(CollectionTest, ReserveIncreasesCapacityNotSize)
{
    // Add entries
    add_entries(20);

    // Verify the reserve increases capacity not size
    size_t prevCapacity = collection->capacity();
    size_t prevSize = collection->size();

    collection->reserve(30);

    ASSERT_TRUE(collection->size() == prevSize);
    ASSERT_TRUE(collection->capacity() > prevCapacity);

}
```

```
[ RUN      ] CollectionTest.ReserveIncreasesCapacityNotSize
[       OK ] CollectionTest.ReserveIncreasesCapacityNotSize (0 ms)
```

Green Pace

Unit Testing Out-of-Range

```
// TODO: Create a test to verify the std::out_of_range exception is thrown when calling at() with an index out of bounds
// NOTE: This is a negative test

TEST_F(CollectionTest, IndexOutOfRange)
{
    // Add entries
    add_entries(5);

    // Verify there is an out of range exception using at()
    EXPECT_THROW(collection->at(20), std::out_of_range);

}
```

```
[ RUN      ] CollectionTest.IndexOutOfRange
[       OK ] CollectionTest.IndexOutOfRange (0 ms)
```

Green Pace

Unit Testing
Refill after Clearing

```
// Positive Test: Verify full Lifecycle

TEST_F(CollectionTest, CanRefillAfterClearing)
{
    // Add entries
    add_entries(5);

    // Clear collection
    collection->clear();

    // Add entries
    add_entries(10);

    // Verify it behaves like a fresh collection
    EXPECT_EQ(collection->size(), 10);
    EXPECT_FALSE(collection->empty());

}
```

```
[ RUN      ] CollectionTest.CanRefillAfterClearing
[       OK ] CollectionTest.CanRefillAfterClearing (0 ms)
```

Green Pace

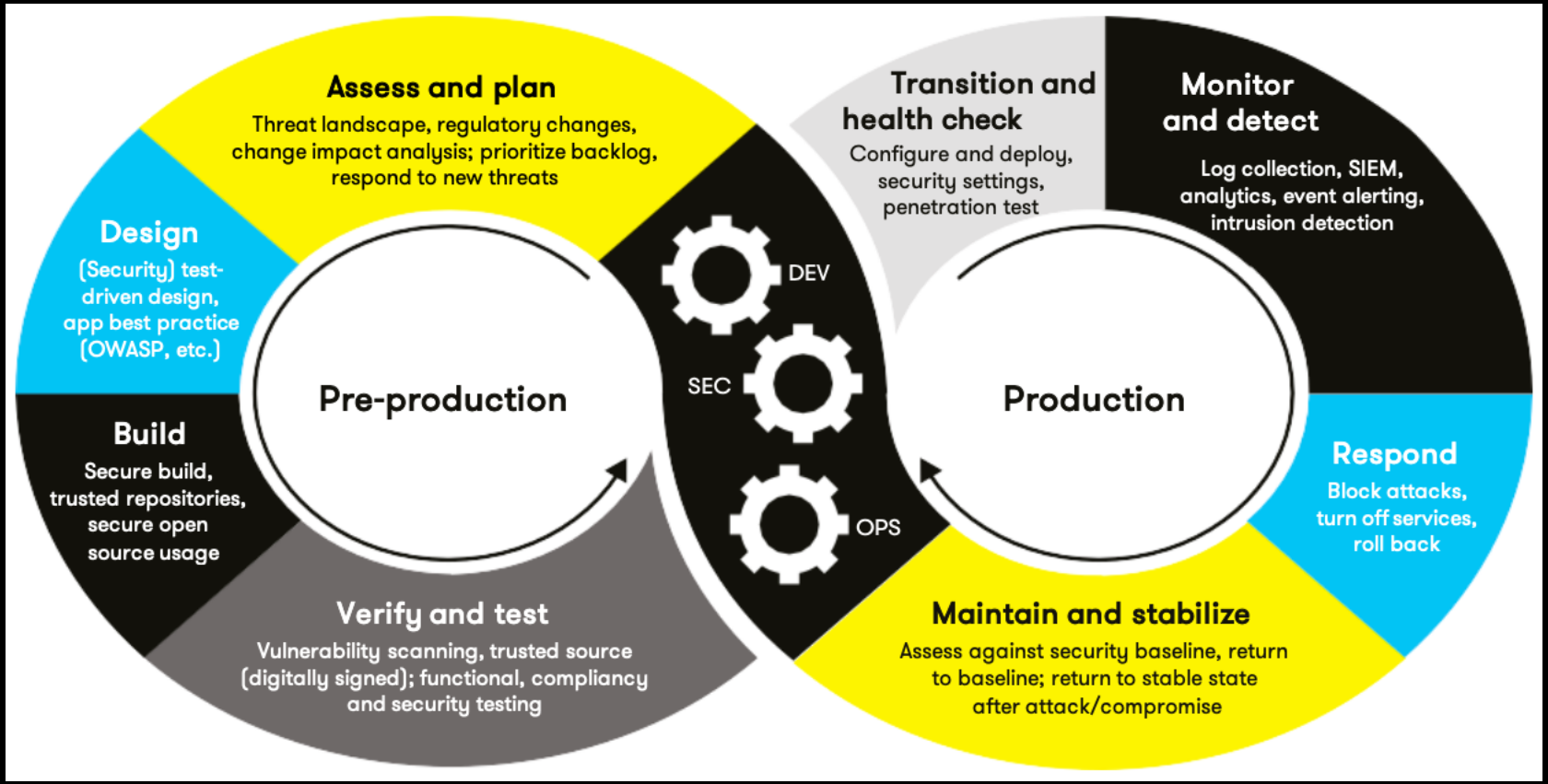
Unit Testing Pop Back

```
// Negative: Verify Pop from empty collection

TEST_F(CollectionTest, TestPopBack)
{
    // Add Entries
    add_entries(3);

    // Verify pop_back
    collection->pop_back();
    ASSERT_FALSE(collection->size() == 10);

}
```

```
[ RUN      ] CollectionTest.TestPopBack
[       OK ] CollectionTest.TestPopBack (0 ms)
```

Green Pace

AUTOMATION SUMMARY



Green Pace

TOOLS

- The DevSecOps pipeline is a continuously evolving system that embeds security throughout both the development lifecycle and the live operation of a product. Rather than treating security as a final step, it is integrated into every stage of the automation flow to ensure ongoing protection and improvement.
- Security tools are integrated across all stages of the pipeline. In design, secure test plans are created to identify risks early. During build, the compiler enforces secure coding and detects issues. In testing, tools like unit testing and static analysis automate security checks. Before deployment, configurations are adjusted in the transition phase. In production, monitoring tools detect and respond to threats, while the final stage evaluates effectiveness and maintains system stability.Security automation is therefore present in the design, build, verify/test, and monitoring stages, with the compiler playing a critical role during the build phase as an early line of defense against vulnerabilities.
- Examples of security tools used in this process include:
  - **Parasoft C/C++test** – A static analysis and testing tool that identifies coding errors, enforces compliance with coding standards, and supports unit testing for C and C++ applications.
  - **Astree** – A static analysis tool designed to detect runtime errors in embedded C programs, particularly useful for proving the absence of certain classes of bugs in safety-critical systems.
  - **Helix QAC** – A static code analysis tool that helps developers identify defects, enforce coding standards, and improve overall code quality in C and C++ projects.

Green Pace

RISKS AND BENEFITS

- “Don’t leave security until the end” is a key best practice in software development. It stresses integrating security throughout the entire coding process to ensure vulnerabilities are identified early rather than overlooked at the final stage. This proactive approach reduces the risk of security breaches and results in a more secure, reliable product.
- Delaying security until the end increases the likelihood of undetected vulnerabilities. It can also lead to higher remediation costs, project delays due to additional fixes, and potential loss of customer trust if issues make it into production. Addressing threats during development helps maintain efficiency and strengthens overall product security.

Green Pace

RECOMMENDATIONS

- This guide acts as an initial foundation for the team.
- Additional standards will be introduced over time.
- Its purpose is to provide a dependable reference for developers.
- Standardized coding improves readability and consistency.
- These guidelines promote a cohesive codebase rather than fragmented work.
- The policy must evolve to address changing threats.
- Ongoing updates are essential to handle new risks and vulnerabilities.
- A future objective is to define coding standards for each of the ten principles.
- The goal is to fully support these principles with appropriate coding standards.

Green Pace

CONCLUSIONS

Testing often to exploit vulnerabilities

- Detect vulnerabilities early on
- Save time and effort for fixing errors
- Reduce the likelihood of exploitation

Focusing on defense in depth policy

- Implementing multiple layers of protection
- Standard coding practices for security
- Input validation, error handling, authentication and authorization

Continually monitor the code

- Identifying security gaps and areas for improvement
- Regularly perform security audits
- Validate the absence of breaches

Green Pace

REFERENCES

SEI CERT C++ Coding Standard | CERT Secure Coding. (2016). https://cmu-sei.github.io/secure-coding-standards/sei-cert-cpp-coding-standard/

Green Pace